

Technical Project Report

AI-Powered Customer Complaint Management System (QMS)

Candidate: Shreeja Hebbar

Role applied for: AI Product Engineer (Fresher / Intern)

Company: AIVOA

Assessment: Round 1 — Full-Stack AI QMS Application

1. Executive Summary

This report documents the design and implementation of an AI-native Customer Complaint Management System built for the pharmaceutical manufacturing domain, spanning both Active Pharmaceutical Ingredients (API) and Finished Dosage Forms (FDF). The core requirement behind this assignment was to remove manual form entry from the Quality Assurance workflow entirely. To that end, the left-hand QMS complaint form in the application is deliberately kept read-only for human operators; the only way to populate, correct, or reason about a complaint record is through conversation with the AIVOA Co-pilot AI Assistant on the right-hand panel, either by typing a message or uploading a supporting document.

The system was built as a full-stack application, with a React front end talking to a FastAPI back end, which in turn hands off reasoning to a LangGraph agent pipeline running on Groq's inference infrastructure. The sections below walk through the architecture, the six tools the agent exposes, the main user workflows, the compliance-oriented features layered on top, and the decisions and trade-offs made along the way.

2. System Architecture & Tech Stack

The application is split into three cleanly separated layers: a React/Redux front end that owns UI state, a FastAPI service that exposes HTTP endpoints and handles validation, and a LangGraph agent pipeline that does the actual language understanding and reasoning. Requests flow from the chat panel down to FastAPI, into the agent graph, and the resulting structured JSON is dispatched back into the Redux store, which re-renders the read-only form on the left. This keeps the UI a pure reflection of agent-produced state rather than something a user edits directly.

Core Technology Stack

Layer	Technology
Frontend	React.js, Redux Toolkit, Axios, Lucide React icon set, Google Inter typeface
Backend	FastAPI (Python), Uvicorn ASGI server, Pydantic for schema validation
AI / Orchestration	LangGraph agent pipeline, LangChain-Groq integration, Groq-hosted Llama-3.3-70b-versatile (gemma2-9b-it as a lighter fallback)
Database	SQLAlchemy ORM over SQLite for local development, with drop-in compatibility for PostgreSQL or MySQL in production

3. Implemented LangGraph AI Tools

The agent exposes six specialised tools, five of which were required by the assessment brief and one added as a bonus. Each tool is a discrete, testable unit that the LangGraph orchestrator can call depending on what it infers the user is trying to do — log a new complaint, correct an existing one, pull data out of an uploaded document, or reason about risk and corrective action.

#	Tool Name	Description & Functional Scope
1	log_complaint_tool	Extracts unstructured text input into the key QMS schema fields — customer, product, batch, dates, and defect description.
2	edit_complaint_tool	Performs state-aware updates on targeted fields (for example, correcting a batch number) while retaining all existing form context.
3	extract_document_tool	Ingests raw text streams from uploaded PDF or plain-text complaint emails and reports, and auto-fills the registry.
4	assess_risk_tool	Evaluates pharma compliance parameters to assign a risk severity level (Critical, Major, Minor) and recommend immediate QA steps.
5	suggest_capa_tool	Formulates 5-Why root cause hypotheses and Corrective and Preventive Actions (CAPA) for the QA team.
6	check_completeness_tool (bonus)	Audits the current form state against mandatory fields and returns a live completeness score, highlighting missing data.

4. Key Functional Workflows

Workflow 1 — Natural Language Complaint Ingestion

- The user types an informal complaint, for example: “Apollo Pharmacy reported discolored capsules in Amoxicillin 500mg, batch BMX24601.”
- FastAPI forwards the message to the LangGraph pipeline.
- The agent runs `log_complaint_tool`, `assess_risk_tool`, and `suggest_capa_tool` in sequence.
- The resulting structured JSON is returned to the front end and dispatched into Redux, which populates the read-only form on the left.

Workflow 2 — Context-Aware Editing

- The user sends a correction, for example: “Sorry, the batch number is BMX24602 and quantity is 48 capsules.”
- The agent classifies this as an edit rather than a new complaint, pulls the current Redux state, and calls `edit_complaint_tool`.
- Only the mentioned fields change; everything else — product name, expiry date, customer name — stays exactly as it was.

Workflow 3 — Unstructured Document Ingestion

- The user uploads a complaint email or QA report (.txt or .pdf).
- The server extracts the raw text and passes it to `extract_document_tool`.
- Lot numbers, packaging specifications, and defect descriptions are parsed out and used to populate the form directly, without any typing.

5. Quality & Compliance Features

- Visual field feedback: React components highlight whichever fields the AI just changed, so the operator can see at a glance what moved.
- Severity badging: a dynamic status indicator reflects the AI-assessed risk level — Critical, Major, or Minor.
- Domain-aware reasoning: the agent generates QA guidance in the language of the domain — quarantine notices, sealing-line audit recommendations, batch replacement authorisations — rather than generic text.

- Live completeness scoring: the bonus `check_completeness_tool` continuously audits the form against mandatory pharmaceutical compliance fields before a record can be saved.

6. Backend Implementation Detail

`backend/agent.py` defines the complaint state schema as a Python TypedDict and implements each of the five core tools as an `@tool`-decorated function. `log_complaint_tool` prompts the Groq LLM to turn free text into a structured dictionary; `edit_complaint_tool` takes both the existing state and the new instruction so it can merge changes selectively instead of overwriting the whole record; `extract_document_tool` is configured to read raw text pulled from uploaded PDFs and plain-text files; `assess_risk_tool` and `suggest_capa_tool` encode the domain-specific reasoning for severity classification, root-cause hypotheses, and CAPA suggestions. All five are wired together inside a single `execute_qms_agent` entry point, which is what FastAPI actually calls.

On the API side, `backend/main.py` exposes three routes, and `backend/database.py` defines the SQLAlchemy models used to persist saved complaints.

Endpoint	Purpose
POST /api/chat	Receives natural-language input from the chat panel and routes it through the LangGraph pipeline.
POST /api/extract-document	Accepts an uploaded file (via <code>python-multipart</code>), parses it, and returns structured field data.
POST /api/save-complaint	Persists a validated complaint record to the database.

Using SQLAlchemy as the ORM layer means the application runs against SQLite with zero configuration during development, but the same models can point at PostgreSQL or MySQL in production without any code changes — only the connection string differs.

7. Frontend Implementation Detail

State management on the client lives in `frontend/src/store.js`, built with Redux Toolkit. Whenever a response comes back from FastAPI, the app dispatches an `updateFormData` action, and every component subscribed to that slice re-renders automatically. In `App.jsx`, every input element on the complaint form is explicitly marked `readOnly`

— this is the one line, functionally, that enforces the entire premise of the assignment: no field can be edited by a mouse click or a keystroke, only by a message to the assistant.

8. Key Design Decisions & Learnings

State preservation during conversational editing. The trickiest part of the build was making sure that a follow-up correction like “change the batch number to BMX24602” didn't wipe out fields the user hadn't mentioned. The fix was to always pass the current Redux state into `edit_complaint_tool` alongside the new instruction, so the LLM merges rather than replaces.

AI-driven UI control flow. Building a UI that is strictly driven by an agent — rather than by direct user input — meant the Redux slices and the agent's JSON output had to stay in lockstep. Any mismatch between what the tool returned and what the reducer expected would silently break the form, so a consistent schema across the agent and the store turned out to matter more than any individual tool's logic.

Domain compliance in pharma QMS. Working through this project meant getting familiar with real pharmaceutical QA conventions — the API vs. FDF distinction, why batch traceability and moisture-related defect indicators matter, and how a 5-Why root-cause analysis actually gets structured. Encoding that domain knowledge into the risk and CAPA tools, rather than leaving them generic, is what made the output feel usable rather than decorative.

9. Repository & Setup Instructions

- GitHub repository: `pharma-qms-ai-complaints`
- Backend: `cd backend && uvicorn main:app --reload` (runs on port 8000)
- Frontend: `cd frontend && npm run dev` (runs on port 5173)
- API documentation is auto-generated by FastAPI and viewable at `/docs` once the backend is running.

10. Conclusion

This project set out to prove a fairly specific constraint — that an entire QA data-entry workflow could be driven end-to-end by an AI agent, with the human operator never touching the form directly — and the final build holds to that constraint throughout. Beyond meeting the five required tools, the completeness-checker and the severity/CAPA reasoning were added to make the tool feel closer to something a real QA team could use, not just a demo. The full source, including the LangGraph agent, the FastAPI service, and the React front end, is available in the linked GitHub repository.